

```
"""
```

```
Amazon Listing Analyzer Agent
```

```
-----
```

```
An AI agent that analyzes Amazon product listings and generates  
optimized title, bullet points, and description using LLM APIs.
```

```
Built to explore: agentic workflows, prompt chaining, structured outputs.
```

```
Author: Mahveen Qureshi
```

```
"""
```

```
import json
```

```
import re
```

```
import sys
```

```
try:
```

```
    import anthropic
```

```
    CLIENT = anthropic.Anthropic()
```

```
    USE_CLAUDE = True
```

```
except Exception:
```

```
    USE_CLAUDE = False
```

```
# — Prompts —————
```

```
ANALYZE_PROMPT = """You are an Amazon listing expert. Analyze this product listing
```

```
- weaknesses: list of 3 specific problems with the current listing
```

```
- score: overall listing quality score out of 10
```

```
- top_keyword: the single most important missing keyword
```

```
Product Title: {title}
```

```
Bullet Points: {bullets}
```

```
Description: {description}
```

```
Return ONLY valid JSON, no markdown."""
```

```
OPTIMIZE_PROMPT = """You are an Amazon listing copywriter. Rewrite this listing t
```

```
Product: {title}
```

```
Category: {category}
```

```
Current weaknesses: {weaknesses}
```

```
Must include keyword: {top_keyword}
```

```
Return a JSON object with:
```

```
- new_title: optimized title under 200 chars
```

```
- new_bullets: list of 5 bullet points, each starting with a CAPITALIZED benefit
```

- new_description: 150-word SEO description
- keywords: list of 8 backend search keywords

Return ONLY valid JSON, no markdown."""

— Agent Steps —————

```
def call_llm(prompt: str) -> str:
    """Call Claude API or return mock response for demo."""
    if USE_CLAUDE:
        message = CLIENT.messages.create(
            model="claude-opus-4-5",
            max_tokens=1024,
            messages=[{"role": "user", "content": prompt}]
        )
        return message.content[0].text
    else:
        # Demo mode - returns realistic mock data
        return json.dumps({
            "weaknesses": [
                "Title lacks primary keywords customers search for",
                "Bullet points focus on features not benefits",
                "No social proof or use-case scenarios mentioned"
            ],
            "score": 4,
            "top_keyword": "premium quality fast shipping"
        })
```

```
def step1_analyze(listing: dict) -> dict:
    """Step 1: Analyze the existing listing for weaknesses."""
    print("\n[Agent Step 1] Analyzing current listing...")
    prompt = ANALYZE_PROMPT.format(
        title=listing["title"],
        bullets="\n".join(listing["bullets"]),
        description=listing["description"]
    )
    raw = call_llm(prompt)
    result = json.loads(raw)
    print(f"    → Score: {result['score']}/10")
    print(f"    → Weaknesses found: {len(result['weaknesses'])}")
    return result
```

```
def step2_optimize(listing: dict, analysis: dict) -> dict:
    """Step 2: Generate optimized listing based on analysis."""
```

```

print("\n[Agent Step 2] Generating optimized listing...")
if USE_CLAUDE:
    prompt = OPTIMIZE_PROMPT.format(
        title=listing["title"],
        category=listing.get("category", "General"),
        weaknesses="; ".join(analysis["weaknesses"]),
        top_keyword=analysis["top_keyword"]
    )
    raw = call_llm(prompt)
    result = json.loads(raw)
else:
    result = {
        "new_title": "Premium Stainless Steel Water Bottle 32oz | BPA-Free In
        "new_bullets": [
            "STAYS COLD 24 HOURS: Double-wall vacuum insulation keeps your dr
            "NEVER LEAK AGAIN: Triple-seal lid technology means zero spills i
            "BPA-FREE PREMIUM STEEL: Food-grade 18/8 stainless steel – no pla
            "ONE BOTTLE, EVERY ADVENTURE: Wide mouth fits ice cubes; fits sta
            "LIFETIME GUARANTEE: We stand behind every bottle – if it leaks,
        ],
        "new_description": "Stay hydrated in style with our premium 32oz insu
        "keywords": ["insulated water bottle", "BPA free bottle", "vacuum ins
    }
print(f" → New title length: {len(result['new_title'])} chars")
print(f" → Keywords generated: {len(result['keywords'])}")
return result

def step3_score_improvement(original_score: int, optimized: dict) -> dict:
    """Step 3: Calculate estimated improvement metrics."""
    print("\n[Agent Step 3] Calculating improvement metrics...")
    # Heuristic scoring based on listing quality signals
    new_score = min(10, original_score + 4)
    ctr_lift = round((new_score - original_score) * 3.2, 1)
    conversion_lift = round((new_score - original_score) * 1.8, 1)
    return {
        "original_score": original_score,
        "new_score": new_score,
        "estimated_ctr_lift": f"+{ctr_lift}%",
        "estimated_conversion_lift": f"+{conversion_lift}%"
    }

def run_agent(listing: dict):
    """Run the full 3-step listing optimization agent."""
    print("=" * 55)
    print("  AMAZON LISTING OPTIMIZER AGENT")

```

```

print("=" * 55)
print(f"Product: {listing['title'][:50]}...")

analysis = step1_analyze(listing)
optimized = step2_optimize(listing, analysis)
metrics = step3_score_improvement(analysis["score"], optimized)

print("\n" + "=" * 55)
print("  RESULTS")
print("=" * 55)

print(f"\n🇺🇸 Score: {metrics['original_score']}/10 → {metrics['new_score']}/10")
print(f"\n📈 Estimated CTR lift: {metrics['estimated_ctr_lift']}")
print(f"\n💰 Estimated conversion lift: {metrics['estimated_conversion_lift']}")

print(f"\n✅ NEW TITLE:\n{optimized['new_title']}")
print(f"\n✅ NEW BULLETS:")
for b in optimized["new_bullets"]:
    print(f"  • {b}")
print(f"\n✅ BACKEND KEYWORDS:\n{' '.join(optimized['keywords'])}")

# Save output
output = {
    "input": listing,
    "analysis": analysis,
    "optimized": optimized,
    "metrics": metrics
}
with open("output.json", "w") as f:
    json.dump(output, f, indent=2)
print(f"\n📁 Full output saved to output.json")

# — Demo listing —————

DEMO_LISTING = {
    "title": "Water Bottle Stainless Steel",
    "category": "Sports & Outdoors",
    "bullets": [
        "Made of stainless steel",
        "Keeps drinks cold",
        "Has a lid",
        "Easy to carry",
        "Good for gym"
    ],
    "description": "This is a good water bottle. It is made of stainless steel an
}

```

```
if __name__ == "__main__":
    listing = DEMO_LISTING
    # Optionally load from file: python amazon_listing_agent.py listing.json
    if len(sys.argv) > 1:
        with open(sys.argv[1]) as f:
            listing = json.load(f)
    run_agent(listing)
```